

ファイル構成・依存関係マップ（レンタルスタジオサーチアプリ）

本書は「どの画面が何をするか」「どのファイルがどのファイルを読んでいるか」を一覧できるようにするための リファレンス。対象を4ブランド（BUZZ／studio worcle／サウンドスタジオNOAH／スタジオミッション）の 実店舗に絞り、AIスコアリング機能を廃止した後の最新構成を反映している。仕様の背景・意思決定理由は AWS構成・サービス連携.html／詳細設計.html／企画・設計アーカイブ.html を参照。

1. 画面一覧（フロントエンド）

URL	画面名	ページファイル	やること
/	トップ （スタジオ 一覧）	pages/TopPage.tsx	4ブランドのスタジオをカード表示。ブランド・エリア・部屋の広さで絞り込み。「空き状況を見る」でAvailabilityModalを開きタイムテーブル表示。「現在地から探す」ボタンも維持
/map	地図	pages/MapPage.tsx	Google Maps上に全スタジオをブランド色分けピン表示（スコアによる色分けは廃止）
/studios	スタジオ 比較	pages/StudiosPage.tsx	スタジオ×部屋の広さを比較できる一覧表（スコア表示は廃止）
/favorites	保存済み	pages/FavoritesPage.tsx	お気に入り登録済みスタジオのみ表示
/events	イベント （新規）	pages/EventsPage.tsx	ログイン必須。「次のイベント」（イベント名・ステージ横幅/奥行き・出演人数）の登録・削除。登録済みイベント・目的（振り入れ/構成）・日付・時間・利用時間を指定して「おすすめを探す」→ 条件を満たすスタジオ・部屋の一覧を表示
/posts	レビュー	pages/PostsPage.tsx	レビュー投稿（★評価・本文・写真）の一覧・作成・編集・削除。編集は本人の投稿のみ、投稿先スタジオは変更不可
/chat	AI相談	pages/ChatPage.tsx	写真添付対応のAIチャット、会話履歴、送信済みメッセージの編集。実データ（Studios）に基づいた回答
/guide	使い方	pages/GuidePage.tsx	画面ごとの使い方・ホーム画面への追加手順・FAQ

ルーティング定義は App.tsx。全ページ共通のヘッダーは components/NavBar.tsx（ロゴクリックで / に戻る、モバイルではラベル非表示、右端にログイン状態を表示。「イベント」タブを新規追加）。

ログイン不要（閲覧）：スタジオ一覧・空き状況・地図・スタジオ比較・レビュー投稿の一覧表示。

ログイン必須（操作）：お気に入り登録/削除、投稿の作成/編集/削除、AI相談、 イベントの登録/削除/おすすめ検索、現在地から探す（Places APIのコスト保護のため）。

未ログインでこれら进行操作しようとするとCognito Hosted UI（Googleログイン画面）へ自動遷移するか（お気に入り・投稿作成・現在地から探す）、ページ自体がログイン案内を表示する（AI相談・イベント）。

「AI分析を実行」ボタンおよびそれに関連するUI（AiBatchButton.tsx、TOP3表示）は スコアリング機能の廃止にともない**削除済み**。

2. フロントエンド ファイル依存関係

2.1 ページ → フック → コンポーネント

ページ	使用するフック	使用する主なコンポーネント
TopPage.tsx	useFavorites (api/client.getStudiosを直接呼び)	StudioCard, StudioDiscoveryButton, DetailModal, NearbyModal, AvailabilityModal
MapPage.tsx	(なし。api/client を直接呼び)	@vis.gl/react-google-maps (外部ライブラリ)
StudiosPage.tsx	(なし。api/client を直接呼び)	ImagePreviewPopover
FavoritesPage.tsx	useFavorites	StudioCard, DetailModal
EventsPage.tsx	(なし。api/client を直接呼び)	DetailModal (おすすめ結果からスタジオ詳細を開く用)
PostsPage.tsx	usePosts	PostForm
ChatPage.tsx	useChat	ChatInput, ChatHistoryPanel
GuidePage.tsx	(なし。静的コンテンツのみ)	(なし)

2.2 コンポーネント間の依存

コンポーネント	使われる場所	内部で使うもの
StudioCard.tsx	TopPage, FavoritesPage, NearbyModal	utils/brand.ts (ブランドラベル・色), ImagePreviewPopover, api/client (recordAnalyticsEvent)
AvailabilityModal.tsx	TopPage (新規)	api/client (getStudioAvailability)。日付選択 + 部屋ごとの30分刻みタイムテーブル表示。データが無い日は「空き状況の情報がありません」と表示
DetailModal.tsx	TopPage, FavoritesPage, NearbyModal, EventsPage	ImagePreviewPopover, api/client (写真アップロード・recordAnalyticsEvent)。部屋一覧に写真・平面図・設備タグ・「この部屋を公式サイトで見る」リンクを表示。予約ボタンはstudio.websiteが無い場合studio.sourceUrlにフォールバック
NearbyModal.tsx	TopPage	hooks/useGeolocation, utils/distance.ts, StudioCard
ImagePreviewPopover.tsx	StudioCard, StudiosPage, DetailModal	hooks/useLongPress (imageUrlのみ表示)
PostForm.tsx	PostsPage	api/client (署名付きアップロード・投稿作成)。★評価入力UIを含む
ChatInput.tsx / ChatHistoryPanel.tsx	ChatPage	(propsで受けたコールバックのみ) ChatInputはhooks/useGeolocationで現在地共有トグルを実装
StudioDiscoveryButton.tsx	TopPage	api/client (runStudioDiscovery) を直接呼び
ProfileModal.tsx	NavBar	(propsで受けたコールバックのみ。API直呼びなし)

削除済みコンポーネント: RecommendationCard.tsx (スコア・TOP3表示用)、AiBatchButton.tsx (「AI分析を実行」ボタン) は、AIスコアリング機能の廃止にともない削除した。StudioCard.tsx が旧 RecommendationCard.tsx の表示ロジックを引き継ぎつつ、スコア表示部分を除いた形になっている。

2.3 フック → APIクライアント

フック	呼び出す api/client.ts の関数
useFavorites.ts	getFavorites, addFavorite, removeFavorite (未ログイン時はfetchせず空配列、トグル時はsigninRedirect)

usePosts.ts	getPosts, createPost, updatePost, deletePost（削除失敗時はdeleteErrorに反映）
useChat.ts	sendChatMessage, editChatMessage, getChatHistory, getChat, deleteChat, getPresignedUploadUrl, uploadImageToS3（429応答はレート制限メッセージとして表示。lat/lngを渡せる）
useProfile.ts	getMyProfile, updateMyProfile（未ログイン時はfetchしない。NavBarから使われる）
useGeolocation.ts	（なし。navigator.geolocation をラップするのみ。TopPage/StudioDiscoveryButton/ChatInputから使われる）
useLongPress.ts	（なし。hover/長押し検知のみのUIフック）

api/client.ts が全エンドポイント呼び出しを一手に引き受ける唯一の窓口（共有axiosインスタンス）。新規追加された関数として getStudioAvailability（GET /studios/{studioId}/availability）、getEvents / createEvent / deleteEvent（/events系）、getRecommendedStudios（GET /recommend-studios）がある。**削除済み:** getRecommendations、runAiBatch（POST /admin/run-ai-batch呼び出し）は、対応するバックエンドAPIの削除にともない削除した。

2.4 認証まわりのファイル

ファイル	役割
auth/authConfig.ts	react-oidc-context の AuthProvider に渡す設定（authority/client_id/redirect_uri）。環境変数 VITE_COGNITO_* から組み立てる。Cognito Hosted UIの独自ログアウトURLを作る cognitoSignOut() もここに定義
auth/AuthTokenSync.tsx	画面には何も描画しない同期用コンポーネント。ログイン状態が変わるたびに api/client.ts の setAuthToken() を呼び、以後のAPIリクエストのAuthorizationヘッダーを最新化する

App.tsx が <AuthProvider> でアプリ全体をラップし、直下に <AuthTokenSync /> をマウントする。各フック・コンポーネントは react-oidc-context の useAuth() を直接呼んでログイン状態を参照する（Reduxのような中央ストアは無く、Contextで完結）。

2.5 純粋ユーティリティ

ファイル	内容	ユニットテスト
utils/brand.ts	4ブランド（BUZZ/worcle/NOAH/ミッション）の一覧（BRANDS）、表示ラベル（BRAND_LABELS）、地図・カードの色分け（BRAND_COLORS）	-
utils/area.ts	スタジオ住所からの都道府県名抽出（エリア絞り込み用）	utils/area.test.ts
utils/distance.ts	haversine公式による2点間距離計算	utils/distance.test.ts
types/index.ts	Studio/Room/StudioAvailability/StudioEvent/Favorite/Post/Chat等、全体で共有する型定義	-

削除済みユーティリティ: utils/score.ts（スコア→色/ラベル変換、距離ベースのスコア近似再計算）は、スコアリング機能の廃止にともない削除した。地図・カードの色分けは utils/brand.ts のブランド別配色に置き換えている。

3. バックエンド ファイル構成（Lambda・ローカルスクリプト）

3.1 API系（backend/lambda/api/handlers.py 1ファイルに集約）

関数	エンドポイント	認証	読み書きするテーブル/バケット
----	---------	----	-----------------

getStudiosHandler	GET /studios	不要	Studios（読、対象4ブランドのみフィルタ）
getStudioAvailabilityHandler	GET /studios/{studioId}/availability	不要	Availability（読。データ無し時はrooms: []/scrapedAt:nullで200）
postAnalyticsEventHandler	POST /analytics/event	不要	AnalyticsEvents（作成）
getEventsHandler	GET /events	必須	Events（読、userIdでquery）
postEventsHandler	POST /events	必須	Events（作成）
deleteEventHandler	DELETE /events/{eventId}	必須	Events（削除）
getRecommendedStudiosHandler	GET /recommend-studios	必須	Events（読）、Studios（読）、Availability（読）。広さ・空き状況の両方を満たす部屋のみ抽出
putStudioImageHandler	PUT /studios/{studioId}/image	必須	Studios（更新）
getPostsHandler	GET /posts	不要	Posts（読）、Users（読、投稿者の表示名を結合）
postPostsHandler	POST /posts	必須	Posts（作成。userIdは_get_user_id(event)で取得しリクエストボディの値は無視）
putPostHandler	PUT /posts/{postId}	必須	Posts（更新。content/imageUrl/ratingのうち指定されたフィールドのみ更新）
deletePostHandler	DELETE /posts/{postId}	必須	Posts（削除。postId単一PKで所有者情報がキーに無いため、削除前にget_itemして自分の投稿か明示チェック。他人の投稿は403）
getFavoritesHandler	GET /favorites	必須	Favorites（読）、Studios（読、結合用）
postFavoritesHandler	POST /favorites	必須	Favorites（作成）
deleteFavoritesHandler	DELETE /favorites/{studioId}	必須	Favorites（削除）
postPresignUploadHandler	POST /uploads/presign	必須	S3 UploadsBucket（署名付きPOSTフォーム発行のみ）
postChatHandler	POST /chat	必須	Chats（読み書き）、Usage（1日あたり利用回数のアトミック加算）、Studios（読、実データグラウンディング用）、S3（画像読み取り）、SSM（Claudeキー）
getChatsHandler	GET /chats	必須	Chats（読、軽量版）
getChatHandler	GET /chats/{chatId}	必須	Chats（読、全メッセージ）
deleteChatHandler	DELETE /chats/{chatId}	必須	Chats（削除）

getMyProfileHandler	GET /me	必須	Users（読）。未登録でも404にせずdisplayName:nullで200を返す
putMyProfileHandler	PUT /me	必須	Users（作成/更新。表示名は1〜30文字でバリデーション）

削除済みハンドラー: `getRecommendationsHandler`（GET /recommendations）は、RecommendationsTableごと削除されたAIスコアリング機能の廃止にともない削除した。

「認証必須」のエンドポイントは `template.yaml` の `Api` リソースに定義した `CognitoAuthorizer`（Cognito User Pool Authorizer）を通過して初めて呼び出せる。通過後のイベントには `event.requestContext.authorizer.claims.sub` にユーザーの一意なID（Cognitoのsub）が入り、`_get_user_id(event)` ヘルパーがこれを取得する（Favorites/Chats/Eventsテーブルは元々 `userId` が `partition key` の一部のため、他人になりすましてアクセスすること自体ができない設計。Postsテーブルだけ `postId` 単一PKのため上記の通り明示チェックが必要）。

3.2 バッチ系（backend/lambda/batch/）

ファイル	役割	他ファイルとの関係
batch_common.py	DynamoDB/SSM/HTTP GETの共有ヘルパー。 <code>get_user_id/check_and_increment_daily_usage</code> （ <code>handlers.py</code> の同名関数と同じロジックの <code>api/</code> 非依存版、コスト保護レート制限用）	<code>discover_studios.py</code> ・ <code>admin_trigger.py</code> から import される（循環import回避のため切り出し）
discover_studios.py	Google Places APIで4ブランドの新規店舗候補を探索しStudiosへ追加する中核ロジック（ <code>run_discovery</code> ）。店名から4ブランドいずれかにマッチしない候補は登録しない（ <code>detect_brand</code> ）	<code>batch_common</code> を import。discoverStudiosBatch Lambdaとして稼働
admin_trigger.py	フロントのボタンから対象バッチをEvent(非同期)invokeする汎用ラッパー。環境変数 <code>RATE_LIMIT_ACTION/RATE_LIMIT_DAILY</code> でコスト保護のレート制限を実装（上限超過時はバッチを起動せず429）	<code>adminRunStudioDiscovery</code> Lambdaとして稼働（ <code>BATCH_FUNCTION_NAME=DiscoverStudiosBatch</code> ）
send_analytics_digest.py	直近24時間の閲覧数・予約クリック数を集計しSESでメール通知	<code>batch_common</code> には依存しない独立モジュール。 <code>sendAnalyticsDigestBatch</code> としてEventBridgeから起動

削除済み: `generate_studio_score.py`（スコア計算+AI推薦理由生成のバッチ本体）は、AIスコアリング機能の廃止にともない完全に削除した。`admin_trigger.py`の`BATCH_FUNCTION_NAME`環境変数で切り替えていた「AI分析を実行」用の設定（`RATE_LIMIT_ACTION=ai-batch`）も削除し、現在は「現在地から探す」用の設定のみが残っている。

3.3 ローカル専用スクリプト（backend/scripts/）

ファイル	役割	実行環境
scrape_availability.py	4ブランドの公式サイトから空き状況・部屋の広さ・写真・平面図・設備をスクレイピングし、StudiosTable（部屋マスタ）とAvailabilityTableへ書き込む新規スクリプト。BeautifulSoup（BUZZ/NOAH/ミッションの一部は静的HTML）とPlaywright（worcle・ミッションはJS描画が必要）を使い分け。ミッションは <code>backend/.env</code> の会員ログイン情報でPlaywrightからログイン	ローカル専用。手動またはローカルのタスクスケジューラで定期実行。クラウドにはデプロイしない
backfill_studio_details.py	旧スコアリング項目（収容人数・最寄り駅距離・料金）を発見済みスタジオへ後追いで埋める一回限りのメンテナンススクリプト（旧構成の遺産。詳細は企画・設計アーカイブ.html参照）	ローカル専用（ <code>--dry-run</code> オプションあり）

delete_yoga_pilates.py	4ブランド限定以前にGoogle Places発見で混入したヨガ・ピラティス系スタジオをStudiosTableから削除する一回限りのクリーンアップスクリプト	ローカル専用
------------------------	---	--------

3.4 環境変数ファイル (backend/)

ファイル	内容
.env.example	backend/.envに必要な環境変数のテンプレート（新規追加）。MISSION_LOGIN_EMAIL・MISSION_LOGIN_PASSWORD（スタジオミッションの会員ログイン情報）等を含む。Gitにコミットされる
.env	実際の会員ログイン情報等を格納するローカル専用ファイル。scrape_availability.pyがPlaywrightでのログインに使用。gitignore対象でコミットされない

3.5 SAM関数 ⇔ ハンドラー対応 (infrastructure/template.yaml、抜粋)

SAM論理ID	Handler指定	トリガー
GetStudiosFunction	handlers.getStudiosHandler	API Gateway (GET /studios)
GetStudioAvailabilityFunction	handlers.getStudioAvailabilityHandler	API Gateway (GET /studios/{studioId}/availability)
GetEventsFunction / CreateEventFunction / DeleteEventFunction	handlers.getEventsHandler / postEventsHandler / deleteEventHandler	API Gateway (/events系)
GetRecommendedStudiosFunction	handlers.getRecommendedStudiosHandler	API Gateway (GET /recommend-studios)
DiscoverStudiosBatch	discover_studios.handler	AdminRunStudioDiscoveryFunction 経路のみ（自動スケジュールなし）
AdminRunStudioDiscoveryFunction	admin_trigger.handler (BATCH_FUNCTION_NAME=DiscoverStudiosBatch, RATE_LIMIT_ACTION=studio-discovery, RATE_LIMIT_DAILY=10)	POST /admin/run-studio-discovery (認証必須)
SendAnalyticsDigestBatch	send_analytics_digest.handler	EventBridge（毎日AM6:30 JST）

削除済みSAMリソース: GenerateStudioScoreBatch・AdminRunAiBatchFunction・GetRecommendationsFunction・RecommendationsTable
は、AIスコアリング機能の廃止にともない template.yaml から完全に削除した。毎日AM6:00 JSTのEventBridgeスケジュール（スコア再計算用）も削除した。

Lambda関数名について: API系・バッチ系とも FunctionName は明示指定せず、CloudFormationに自動採番させている。当初は釣行AIアプリと同じ命名で固定していたが、Lambda関数名はアカウント+リージョンで一意という制約に反して既存の釣行AIアプリの同名関数と衝突し、初回デプロイが失敗した経緯がある（AWS構成・サービス連携.html参照）。

4. DynamoDBテーブル一覧

物理名	論理ID	キー構成	書き込み元
studio-studios	StudiosTable	studioId (PK)	discover_studios.run_discovery, scrape_availability.py（部屋マスタ）, putStudioImageHandler, backfill_studio_details.py（手動、旧構成の遺産）

studio-availability	AvailabilityTable	studioId (PK) + date (SK)	scrape_availability.py (ローカルから直接put_item)
studio-events	EventsTable	userId (PK) + eventId (SK)	postEventsHandler / deleteEventHandler (新規テーブル)
studio-favorites	FavoritesTable	userId (PK) + studioId (SK)	postFavoritesHandler / deleteFavoritesHandler
studio-posts	PostsTable	postId (PK)	postPostsHandler / putPostHandler / deletePostHandler
studio-chats	ChatsTable	userId (PK) + chatId (SK)	postChatHandler / deleteChatHandler
studio-usage	UsageTable	userId (PK) + dateKey (SK, 例: "studio-discovery#2026-07-30")	admin_trigger.py / postChatHandler内の check_and_increment_daily_usage (ADDによるアトミック加算)。TTL (expiresAt) で翌々日ごろ自動削除
studio-users	UsersTable	userId (PK)	putMyProfileHandler (表示名の作成/更新)。 getPostHandlerがscanして投稿者名の結合に使う
studio-analytics-events	AnalyticsEventsTable	eventId (PK)	postAnalyticsEventHandler。TTL (expiresAt) で自動削除、 sendAnalyticsDigestBatchが読み取り集計

削除済みテーブル: `studio-recommendations` (RecommendationsTable) は、AIスコアリング機能の廃止にともない完全に削除した。

S3バケット (`studio-search-app-uploads-<アカウントID>`) はスタジオ写真・投稿写真・チャット添付画像のアップロード先。書き込みは署名付きPOSTフォーム経由のみ、読み取り (GetObject) のみ公開許可。フロント配信用バケット (`studio-search-app-frontend-<アカウントID>`) は別物で、CloudFrontのOrigin Access Control経由でのみ読み取り可能な非公開バケット。

5. データの流れ (代表的なシナリオ)

5.1 トップ画面での空き状況確認

TopPage.tsx (マウント時)

- api/client.ts の getStudios() → GET /studios
- getStudiosHandler (4ブランドのみフィルタして返却)
- StudioCard一覧を表示 (ブランド・エリア・広さで絞り込み可能)
- 「空き状況を見る」クリック
- AvailabilityModal.tsx が開き、日付選択
- api/client.ts の getStudioAvailability(studioId, date) → GET /studios/{studioId}/availability
- getStudioAvailabilityHandler (AvailabilityTableをget_item。無ければ空データを返す)
- 部屋ごとの30分刻みタイムテーブルを表示

5.2 「次のイベント」登録とおすすめ検索

EventsPage.tsx (イベント登録フォーム)

- api/client.ts の createEvent() → POST /events
- postEventsHandler → EventsTableへ保存

EventsPage.tsx (イベント選択+目的・日付・時間・利用時間を指定して「おすすめを探す」)

- api/client.ts の getRecommendedStudios() → GET /recommend-studios
- getRecommendedStudiosHandler
 - EventsTableから対象イベント取得 → 目的別の必要面積 (㎡/人) を算出
 - StudiosTable全件scanし、広さ条件を満たすroomを抽出
 - AvailabilityTableで指定日時のslotsを確認し、空きのあるroomのみ残す
- 条件を満たす {studio, room} の一覧をEventsPageに表示 (スコア付けはしない)

5.3 空き状況データの収集 (ローカル、クラウドを介さない)

開発者PCで python backend/scripts/scrape_availability.py を実行（手動 or タスクスケジューラ）

- 4ブランドそれぞれの公式サイトへアクセス
 - ・BUZZ/NOAH: BeautifulSoupで静的HTMLを解析（NOAHは一部Playwright併用）
 - ・worcle/ミッション: Playwrightでブラウザを自動操作しJS描画後のDOMを取得（ミッションはbackend/.envのMISSION_LOGIN_EMAIL/PASSWORDで事前ログイン）
- 空き状況（30分刻み）・広さ・写真・平面図・設備を取得
- ローカルのAWS認証情報でStudiosTable（部屋マスタ）・AvailabilityTableへ直接書き込み（Lambda/API Gatewayを経由しない）

5.4 チャットで写真つきの質問をしたとき

ChatInput.tsx（写真選択 + 送信）

- useChat.ts の send()
 - api/client.ts の getPresignedUploadUrl() → POST /uploads/presign
 - api/client.ts の uploadImageToS3()（S3へ直接POST、バックエンドを経由しない）
 - api/client.ts の sendChatMessage() → POST /chat
- postChatHandler（handlers.py）
 - S3から画像バイトを取得 → Claudeへマルチモーダル入力として送信
 - Chatsテーブルへ保存 → 応答を同期的に返却

5.5 「初心者でも通えるスタジオある？」とAI相談で聞いたとき

ChatInput.tsx（現在地共有トグルON + 送信）

- useChat.ts の send()（lat/lngを含めてsendChatMessage呼び出し）
- POST /chat
- postChatHandler（handlers.py）
 - _build_studios_context(lat, lng) がStudiosをscan→
現在地からの距離（haversine）で近い順にソート→上位N件をテキスト化（旧来のスコア・reasonは含まない。Studiosの実データのみ）
 - _generate_chat_reply() のsystem_instructionに上記テキストを埋め込む（「データに無いスタジオは作り出さない」という指示も併記）
 - Claudeが実データの中から回答を組み立てる

6. 使用サービスとコストの目安

釣行AIアプリと同じAWSアカウント上で運用しているため、AWS側の各サービスは既存分と合算で 無料枠・低ボリューム内に収まる想定。**正確な最新料金は各サービスの公式ページで確認すること**（料金体系は変更されうるため、あくまで目安）。AI スコアリング機能の廃止にともない、 Claude API・Google Places APIの呼び出し頻度は旧構成よりさらに低くなっている（AI相談チャットと「現在地から探す」時のみ）。

サービス	課金対象	個人利用規模での目安
Lambda	呼び出し回数・実行時間	無料枠内（月100万回・40万GB秒まで無料）
API Gateway (REST)	APIコール数	この規模ならごく僅か
DynamoDB（8テーブル）	読み書きリクエスト数・ストレージ	無料枠+低ボリュームでごく僅か
S3（静的ホスティング+アップロード）	ストレージ・リクエスト数	数百円未満想定
CloudFront	データ転送量	低ボリュームなら僅か
SSM Parameter Store	Standard利用なら無料	無料
EventBridge（日次バッチ起動）	ルール数・イベント数	無料枠内
Cognito	月間アクティブユーザー数	無料枠内

AWS Budgets	予算数	2つまで無料。今回1つのみ設定
Anthropic Claude API (Haiku)	トークン数（画像添付は増加）	AI相談チャットのみで使用。無料枠は無く完全従量課金だがHaikuは低単価。レート制限で抑制済み
Google Places API	リクエスト数（Text Search）	「現在地から探す」実行時のみ課金。既存スタジオでは発生しない
Google Maps JavaScript API	読み込み回数	低ボリューム
Cloudflare Workers（ミラー）	リクエスト数	無料枠（1日10万リクエスト）で十分

AWS側は `CostBudget`（AWS Budgets、月\$10・80%/100%到達時にメール通知）で自動監視している。Google Cloud側（Places/Maps）とAnthropic側（Claude）はAWS Budgetsの対象外のため、それぞれのコンソールで別途予算アラート・spend limitを手動設定する必要がある（README.md参照）。

7. 関連ドキュメント

- 企画の背景・方針転換の経緯: [企画・設計アーカイブ.html](#)
- AWS構成・サービス連携の詳細: [AWS構成・サービス連携.html](#)
- Lambda単位の詳細仕様: [詳細設計.html](#)
- 非技術者向けの全体像: [アプリまるわかりガイド.html](#)